

Engine-Out Case Study

Apollo LM Powered Descent with One DPS Engine Failed

29 July 2026

Contents

1	Engine-Out Case Study	1
1.1	The headline result	2
1.2	Case 2: why there is no solution	2
1.3	The critical engine spacing	3
1.4	Case 3: what an engine-out landing costs	3
1.5	The design trade this exposes	4
1.6	What the model does and does not capture	4

1 Engine-Out Case Study

Three runs of the `apollo_full.py` optimal control problem, identical in every respect except the engine configuration.

#	Directory	Configuration	Result
1	01_nominal_2_engines/	both engines healthy, $y_{eng} = 1.5$ m	lands — 173 iterations, 40 s
2	02_engine_out_nominal_engine_2_roll/	engine 2 roll, $y_{eng} = 1.5$ m	no solution — uncontrollable in roll
3	03_engine_out_survivable_engine_2_roll/	engine 2 roll, $y_{eng} = 0.2$ m	lands — 190 iterations, 76 s

Each directory holds `states.png`, `controls.png`, `actuators.png`, `thrusters.png`, `trajectory_with_axes.png`, the full solver transcript (`console_log.txt`), and the raw arrays (`solution.npz`).

Two cross-case figures sit at the top level:

- `roll_authority_budget.png` — why case 2 cannot work, and where the boundary is
- `nominal_vs_engine_out.png` — case 1 against case 3, on the quantities the failure actually changes

Reproduce with:

```
python run_case_study.py          # all three cases, about 4 min
python plot_roll_authority.py     # analytic budget figure
python plot_comparison.py        # case 1 vs case 3 overlay
```

1.1 The headline result

An engine-out landing is impossible at the vehicle’s designed engine spacing, and the limit is a roll-authority limit, not a thrust limit.

Losing one of two engines halves the available thrust — from 45 040 N to 22 520 N against a 12 530 N hover requirement, so thrust-to-weight only falls from 3.59 to 1.80. That is not what kills the landing. What kills it is that the surviving engine is now 1.5 m off the centreline and its thrust vector no longer passes near the CG, so it applies a **permanent roll moment** that the vehicle has no way to cancel.

1.2 Case 2: why there is no solution

With the survivor carrying hover trim, $T = 12\,530$ N, the roll moment about the CG is

$$L = T \cos \delta_p (- y_{eng} \cos \delta_y + d_{z,eng} \sin \delta_y).$$

The first term is the offset thrust; the second is what the yaw gimbal can buy back by tilting the thrust vector laterally against its own 2.5 m vertical arm. Evaluating the budget at $y_{eng} = 1.5$ m:

Term	Value
Roll moment to cancel ($\delta_y = 0$)	18 796 N · m
Gimbal can cancel (at its 6° limit)	3 293 N · m
RCS can cancel (all 16 thrusters, best case)	2 140 N · m
Deficit	13 363 N · m

The residual 13.4 kN · m acting on $I_{xx} = 5368$ kg · m² is **2.49 rad/s²**, or 143 °/s² of roll acceleration, against an attitude-rate constraint of 10 °/s. The constraint is violated within about 0.07 s. No control history exists that satisfies the problem, so the NLP is genuinely infeasible — this is not a solver-tuning issue.

IPOPT behaves accordingly: it exits with **Maximum_Iterations_Exceeded** after 400 iterations with a **constraint violation of 2.62** still outstanding (cases 1 and 3 both converge to 10^{−6}). The figures in that directory are the final non-converged iterate, plotted for diagnosis and clearly **not a flyable trajectory** — they show the vehicle pinned against every attitude limit at once:

Final iterate, case 2	
Roll ϕ	44.8° (at the 45° limit)
Body rates (p, q, r)	(9.98, 9.92, −9.94) °/s — all three saturated
Position error	1 137 m
Speed	49.0 m/s
Altitude	568 m (never got down)

That is a tumbling vehicle being flung away from the pad, which is the correct physical picture of an untrimmable roll moment.

The iteration cap of 400 for this case is deliberate (`max_iter=400` in `run_case_study.py`); left at the 5000 default IPOPT grinds through restoration phases for a very long time to reach the same conclusion.

1.3 The critical engine spacing

Setting required equal to available and solving for the spacing gives the design boundary:

$$y_{crit} = d_{z,eng} \tan \delta_{y,max} + \frac{L_{RCS,max}}{T_{hover}} = 2.5 \tan 6^\circ + \frac{2140}{12\,530} = 0.263 + 0.171 = \mathbf{0.434\,m}$$

Single-engine-out flight is possible only for $y_{eng} \leq 0.434$ m. The design value of 1.5 m is **3.5× beyond** that. `roll_authority_budget.png` plots this sweep with both cases marked.

Case 3 uses $y_{eng} = 0.2$ m, chosen to sit inside the *stricter* bound $y_{eng} \leq d_{z,eng} \tan \delta_{y,max} = 0.263$ m, where the **gimbal alone** can trim the asymmetry. That leaves the RCS free for manoeuvring rather than spending its entire roll authority holding static trim — a much healthier design point than merely clearing 0.434 m.

The prediction is confirmed by the solve: static trim at 0.2 m needs $\delta_y = \arctan(0.2/2.5) = 4.57^\circ$, and the converged case-3 solution parks the survivor’s yaw gimbal at a mean deflection of **4.65°** — visible as the flat orange shelf in the yaw-gimbal panel of `nominal_vs_engine_out.png`.

1.4 Case 3: what an engine-out landing costs

Case 3 lands successfully — 1 m altitude over the pad, 0.001 m/s, 0.05° attitude error. But the descent is materially different:

Quantity	Nominal	Engine-out ($y_{eng} = 0.2$ m)
Peak total DPS thrust	45 036 N	22 520 N (saturated)
Time to reach contact altitude	about 26 s	about 47 s
Mean survivor yaw-gimbal deflection	1.21°	4.65° (of 6° available)
Total RCS impulse	90 797 N · s	171 124 N · s (+ 88 %)
Terminal speed	0.000 m/s	0.001 m/s
Terminal horizontal error	0.00 m	0.00 m

Three things to take from this:

1. **The descent is far more gradual.** With thrust saturated at the single-engine limit for the whole braking phase, the vehicle needs about 47 s rather than about 26 s to reach contact — it uses most of the fixed 80 s horizon. A more aggressive scenario, or a shorter horizon, would run out of margin.
2. **RCS propellant use nearly doubles.** The RCS is saturated for roughly twice as long, because it is doing attitude control *and* helping hold an asymmetry that never goes away. In a mass-tracking model this would be the binding constraint.

3. **The gimbal spends 78 % of its range on static trim.** 4.65° of the available 6° is consumed just holding the vehicle straight, leaving only about 1.35° for manoeuvring. This is the quantity to watch if the spacing is traded upward.

1.5 The design trade this exposes

The lateral offset is not a free parameter — it is doing useful work in the nominal case. From the baseline `actuators.png`, differential throttling at $y_{eng} = 1.5$ m generates transient roll moments of $\pm 4\,000\text{ N}\cdot\text{m}$, roughly double the entire RCS roll authority of $2\,140\text{ N}\cdot\text{m}$. Shrinking the spacing to 0.2 m cuts that by $7.5\times$ and hands the roll axis back to the RCS.

So the two requirements pull in opposite directions:

- **large** $y_{eng} \rightarrow$ strong differential-throttle roll authority when both engines are healthy, but no engine-out survivability;
- **small** $y_{eng} \rightarrow$ engine-out survivable, but roll control depends on the RCS, which is the propellant-limited actuator.

At $y_{eng} \leq 0.263$ m the vehicle is single-engine-out survivable; above 0.434 m it is not survivable at all. Anything in between survives only by committing the RCS entirely to static trim.

Note also that at $y_{eng} = 0.2$ m the two engines are only 0.4 m apart, which raises plume-impingement and structural-packaging questions this model does not address.

1.6 What the model does and does not capture

The failure is modelled as a **clean, instantaneous, total loss** of engine 2 at $t = 0$, held for the whole horizon: all five of its actuator states and all three of its commands are pinned to zero (`Scenario.failed_eng`). The remaining engine and the RCS are unchanged.

Not modelled, and each would make the real case worse:

- **Failure partway through the descent.** The fault is present from $t = 0$, so the optimiser plans around it with full knowledge. A failure at, say, $t = 20$ s would be a transient recovery problem from a much worse state, and the open-loop OCP cannot represent the detection delay.
- **Failure transients.** No thrust decay tail, no residual thrust, no gimbal hardover — a real engine failure is rarely this tidy.
- **Constant mass and inertia.** As in the baseline model, so the CG does not shift as propellant drains — and a CG offset would change the roll-trim arithmetic directly.
- **Any failure other than total thrust loss** (stuck gimbal, partial throttle loss), which would each need their own bound pattern.
- **Feedback.** These are open-loop reference trajectories; an engine-out descent flown against disturbances would need the tracking controller that sits downstream of this planner.

Above all, case 3 changes the vehicle geometry rather than recovering the as-designed vehicle. It answers “*what spacing would have survived?*”, not “*can the vehicle as built survive?*” — the answer to the second question is no.